RECOLETA TREND BRIEF

代码智能体走向工程闭环：训练、测试与安全验证同时前移

Day trend brief with overview, must-read papers, and cluster notes.

7 tracked topics

WINDOW
Day

PERIOD
2026-03-09 (2026-03-09 to 2026-03-10)

TOPICS
7

OVERVIEW

今天的主题很集中：代码智能体开始真正进入工程闭环，但“可验证”比“更强”更重要。训练、测试、提示词架构和安全门控都在被软件工程化，说明行业正从展示能力转向约束能力。一个突出方向是，代码代理开始更认真地面对真实世界噪声。SWE-Fuse不再假设issue描述可靠，而是把无issue的调试轨迹也纳入训练，直接提升真实修复成绩。

EVOLUTION

3 signals　3 history windows

本窗口延续了代码与安全两条主线，但变化很明确：研究正在把 agent 本身当作工程制品处理。与 软件工程代理走向执行闭环，基础设施与可靠性评测同步升温 (2026-03-07) 的执行闭环相比，今天更强调训练数据、测试编译、提示词审计和安全门控。与 结构化代码智能、长时智能体与 Agent 安全前移 (2026-03-08)、代码智能体走向自纠错、级联部署与可验证安全 (2026-03-06) 的安全治理线索相比，验证对象也从外部行为进一步深入到 system prompt 与多轮迭代过程。

软件工程代理闭环继续推进，但重心上移到训练范式

Continuing

HISTORY

- 结构化代码智能、长时智能体与 Agent 安全前移 (2026-03-08)
- 软件工程代理走向执行闭环，基础设施与可靠性评测同步升温 (2026-03-07)

与 软件工程代理走向执行闭环，基础设施与可靠性评测同步升温 (2026-03-07) 中 Echo、Modulus 所代表的“代理深入软件工程执行链路”一致，今天的 SWE-Fuse 继续把能力重心放在真实仓库修复闭环上，但从流程编排进一步转向训练数据与强化学习设计。它不再只优化执行链路，而是直接处理 issue 文本失真：SWE-smith 中有 30.49% 样本 issue 为空，SWE-Fuse 因此混合 issue-free 轨迹并在 SWE-bench Verified 上把 32B 解题率推到 60.2%，TTS@8 后到 65.2%。这说明代码代理主线仍在延续，只是优化层次从“如何接工具”上移到了“如何在脏监督下学会调试”。

Agent 安全从故障理解转向内部制品验证

Shifting

HISTORY

- 结构化代码智能、长时智能体与 Agent 安全前移 (2026-03-08)
- 代码智能体走向自纠错、级联部署与可验证安全 (2026-03-06)

结构化代码智能、长时智能体与 Agent 安全前移 (2026-03-08) 强调“安全前移”，代码智能体走向自纠错、级联部署与可验证安全 (2026-03-06) 也强调“AI 编码安全转向可验证治理”。今天这一主题明显继续，但焦点从外部故障解释和治理框架，转向更细的内部制品审计。Arbiter 把 system prompt 当作软件架构，在 Claude Code、Codex CLI、Gemini CLI 上共找出 152 个问题，且 Claude Code 的 21 类干扰里有 20 类可静态检测；SCAFFOLD-CEGIS 则把多轮改代码中的安全退化量化为 43.7%，再用语义锚点把它压到 2.1%。相比 代码智能体走向自纠错、级联部署与可验证安全 (2026-03-06) 的故障分类与解释，这一窗口更强调“提示词和迭代过程本身就是待验证对象”。

Agent 的开发与评测流程开始被基准化

Emerging

HISTORY

- 结构化代码智能、长时智能体与 Agent 安全前移 (2026-03-08)
- 软件工程代理走向执行闭环，基础设施与可靠性评测同步升温 (2026-03-07)

相较 结构化代码智能、长时智能体与 Agent 安全前移 (2026-03-08) 的“长时智能体与可监督运行”以及 软件工程代理走向执行闭环，基础设施与可靠性评测同步升温 (2026-03-07) 的协议化基础设施，今天出现了更强的“agent 开发流程软件化”信号。TDAD 将规格编译成测试，再编译回提示词，在 24 次试验里取得 v1 92% 编译成功率、v2 58%，并报告 97% 的回归安全分数；PostTrainBench 则把后训练自动化做成 28 种配置的可重复 benchmark，最佳 agent 均分 23.2%，但仍远低于官方 51.1%。这类工作不再只讨论 agent 如何运行，而是开始把 agent 的构建、评测和升级流程做成基准化工程对象。

CLUSTERS

代码智能体进入真实软件工程闭环

代码智能体的主线继续从“会写代码”转向“能在真实工程约束里闭环执行”。SWE-Fuse 直接处理 issue 描述失真问题，把有描述与无描述轨迹混训，并用熵感知 RLVR 提升真实修复表现；在 SWE-bench Verified 上，8B/32B 解题率分别到 43.0% 与 60.2%，TTS@8 后到 49.8% 与 65.2%。另一侧，微服务生成研究显示代理已能生成可运行服务，但仍未摆脱人工监督：增量重建的单测通过率约 50%–76%，从需求出发的集成测试通过率为 81%–98%。这说明工程代理正在进入“部分可用、但仍需强验证”的阶段。

REPRESENTATIVE SOURCES

- SWE-Fuse: Empowering Software Agents via Issue-free Trajectory Learning and Entropy-aware RLVR Training — Xin-Cheng Wen; Binbin Chen; Haoxuan Lan; Hang Yu; Peng Di; Cuiyun Gao
- Can AI Agents Generate Microservices? How Far are We? — Bassam Adnan; Matteo Esposito; Davide Taibi; Karthik Vaidhyanathan

从提示工程转向可测试的 Agent 规约

今天大量工作把 agent 当成可编译、可测试、可审计的软件制品，而不是一次性 prompt。TDAD 用规格先生成测试，再反向编译提示词；在 24 次试验里，v1 编译成功率 92%，隐藏测试通过率 97%，但规格演化到 v2 后成功率降到 58%。Arbiter 则把 system prompt 当作架构工件分析，在三家编码代理上共找出 152 个问题，Claude Code 单家就有 116 个；其定向分析里 21 类干扰中 20 类可静态检测。共同趋势是：agent 开发正快速借用软件工程里的测试、静态分析和架构审计方法。

REPRESENTATIVE SOURCES

- Test-Driven AI Agent Definition (TDAD): Compiling Tool-Using Agents from Behavioral Specifications — Tzafrir Rehan
- Arbiter: Detecting Interference in LLM Agent System Prompts — Tony Mason

安全治理前移到迭代与系统提示层

安全不再只是外层拦截，而是被嵌进迭代式生成流程本身。SCAFFOLD-CEGIS 指出，在 GPT-4o 的 10 轮迭代中，43.7% 的链条会比初始版本更不安全；即便加显式安全提示，退化率仍有 28.6%。其方法把安全要求变成语义锚点和四层门控，将潜在安全退化率降到 2.1%，并实现 100% 的 safety monotonicity。与 Arbiter 一起看，今天的安全研究重点已从“挡输入攻击”进一步转向“约束 agent 内部演化与系统提示结构”。

REPRESENTATIVE SOURCES

- SCAFFOLD-CEGIS: Preventing Latent Security Degradation in LLM-Driven Iterative Code Refinement — Yi Chen; Yun Bian; Haiquan Wang; Shihao Li; Zhe Cui
- Arbiter: Detecting Interference in LLM Agent System Prompts — Tony Mason

Agent 开始进入研发自动化与测试基础设施

除了直接做代码任务，研究开始评估 agent 能否自动改进模型与测试基础设施。PostTrainBench 把后训练自动化做成受限算力基准：最佳系统 Claude Opus 4.6（Claude Code）加权平均 23.2% ± 1.8，明显高于基础模型 7.5%，但仍低于官方 instruction-tuned 的 51.1%；在 BFCL 上可到 75.9% ± 17.8，甚至在部分模型组合上超过官方调优版本。Java fuzzing 工作则用五智能体流水线生成 harness，在 6 个库、7 个目标方法上取得方法定向覆盖率中位数 +26%，平均每个 harness 约 10 分钟、3.20 美元，并在 12 小时 fuzzing 中发现 3 个未报告 bug。两者都说明 agent 正被用于更上游的研发自动化。

REPRESENTATIVE SOURCES

- PostTrainBench: Can LLM Agents Automate LLM Post-Training? — Ben Rank; Hardik Bhatnagar; Ameya Prabhu; Shira Eisenberg; Karina Nguyen; Matthias Bethge; …
- Coverage-Guided Multi-Agent Harness Generation for Java Library Fuzzing — Nils Loose; Nico Winkel; Kristoffer Hempel; Felix Mächtle; Julian Hans; Thomas Eisenbarth

语言级 Agent 基础设施继续试探

还有一条较弱但值得记录的支线，是把 agent 能力写进语言与运行时。Agentis 把 LLM 设为“标准库”，引入 validate、fuel 与分支探索，并用内容寻址 DAG 管理代码。它代表了语言级 agent 基础设施的探索方向，但当前缺少正式基准与同行评测，证据强度明显弱于当天的论文工作。

REPRESENTATIVE SOURCES

- Agentis — An AI-native programming language where the LLM is the stdlib — ylohnitram